

# 04 February 2026

## Key Highlights and Workflow

- **Project Setup:**
  - Creation of a new [ASP.NET](#) Core MVC project targeting version 9.0+ in Visual Studio 2022.
  - Default folder structure includes Models, Views, Controllers; a new folder named Data is added for DB context.
- **Model Creation:**
  - A Student class is created with properties:
    - StudentId (int, key, required)
    - Name (string, required)
    - Email (string)
    - Course (string)
    - EnrollmentDate (DateTime)
  - Use of **data annotations** like [Key], [Required], and [Display(Name = "...")] to enforce validation and control UI display.
- **Entity Framework Core Setup:**
  - Installation of necessary NuGet packages:

| Package Name                            | Purpose                  |
|-----------------------------------------|--------------------------|
| Microsoft.EntityFrameworkCore           | Core EF functionality    |
| Microsoft.EntityFrameworkCore.SqlServer | SQL Server provider      |
| Microsoft.EntityFrameworkCore.Tools     | EF migration and tooling |
| Microsoft.EntityFrameworkCore.Design    | Design-time support      |

- Creation of ApplicationDbContext inheriting from DbContext.

- Configuration of DbSet<Student> for database operations.
- Registration of DB context in Program.cs with connection string referencing SQL Server.
- **Database Configuration:**
  - Addition of connection string in appsettings.json with:
    - Server name
    - Database name (ASPNetCoreCRUDDb)
    - Trusted connection and server certificate options.
  - Execution of EF Core migrations:
    - Add-Migration Initial
    - Update-Database
  - Verification of database and table creation in SQL Server Management Studio.
- **Scaffolding Controller and Views:**
  - Use of Visual Studio scaffolding to generate:
    - Controller (StudentsController)
    - Views (Create, Edit, Delete, Details, Index) automatically based on Student model and ApplicationDbContext.
  - Explanation of generated views:
    - Index displays a table with student records and action buttons (Edit, Details, Delete).
    - Create and Edit provide forms for data entry and modification.
    - Delete shows a confirmation page.
    - Data annotations affect display labels on forms and lists.
- **UI Enhancements:**
  - Addition of a navigation link to the “Manage Students” section in the shared layout to access the student module.

- **Demonstration of CRUD Operations:**

- Adding new students.
- Editing existing student records.
- Viewing details of a student.
- Deleting student records.
- Changes reflected immediately in the underlying SQL Server database.

# HOW TO GUIDE

Below is your overview rewritten as **step-by-step instructions for students**, with practical code snippets included.

---

## ✅ Step-by-Step Guide: ASP.NET Core MVC CRUD with EF Core (.NET 9+)

---

### ● Step 1: Create a New ASP.NET Core MVC Project

1. Open **Visual Studio 2022**.
2. Click **Create a new project**.
3. Select **ASP.NET Core Web App (Model-View-Controller)**.
4. Click **Next**.
5. Name the project (e.g., StudentManagementApp).
6. Choose:
  - **.NET 9.0**
  - Authentication: None
7. Click **Create**.

### ✓ Verify Default Folder Structure

You should see:

Controllers

Models

Views

wwwroot

**+ Add a New Folder**  On the project folder

1. Right-click the project.
2. Click **Add → New Folder**

3. Name it:

Data

---

## ● Step 2: Create the Student Model

1. Right-click **Models**

2. Click **Add → Class**

3. Name it:

Student.cs

### 🔧 Add the Following Code:

```
using System;
```

```
using System.ComponentModel.DataAnnotations;
```

```
namespace StudentManagementApp.Models
```

```
{
```

```
    public class Student
```

```
    {
```

```
        [Key]
```

```
        [Required]
```

```
        public int StudentId { get; set; }
```

```
        [Required]
```

```
        [Display(Name = "Full Name")]
```

```
        public string Name { get; set; }
```

```
        [Display(Name = "Email Address")]
```

```
        public string Email { get; set; }
```

If you want, you can add the display property for student ID.  
E.g., [Display(Name = "StudentID")]

```
public string Course { get; set; }

[Display(Name = "Enrollment Date")]
public DateTime EnrollmentDate { get; set; }
}
}
```

### What You Learned

- [Key] → Marks primary key
- [Required] → Makes field mandatory
- [Display(Name = "...")] → Controls label text in UI

---

## Step 3: Install Entity Framework Core Packages

Open **Tools** → **NuGet Package Manager** → **Manage NuGet Packages for Solution**

Install:

| Package                                 | Purpose               |
|-----------------------------------------|-----------------------|
| Microsoft.EntityFrameworkCore           | Core EF functionality |
| Microsoft.EntityFrameworkCore.SqlServer | SQL Server provider   |
| Microsoft.EntityFrameworkCore.Tools     | Migrations support    |
| Microsoft.EntityFrameworkCore.Design    | Design-time services  |

---

## Step 4: Create ApplicationDbContext

1. Right-click **Data**
2. Click **Add** → **Class**
3. Name it:

ApplicationDbContext.cs

#### ✎ Add the Following Code:

```
using Microsoft.EntityFrameworkCore;
```

```
using StudentManagementApp.Models;
```

```
namespace StudentManagementApp.Data
```

```
{
```

```
    public class ApplicationDbContext : DbContext
```

```
    {  
         Type 'ctor' and press the TAB key for the shortcut
```

```
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
```

```
            : base(options)
```

```
    {
```

```
    }
```

```
        public DbSet<Student> Students { get; set; }
```

```
    }
```

```
}
```

---

#### ● Step 5: Configure Connection String

Open **appsettings.json**

Add:

```
"ConnectionStrings": {
```

```
    "DefaultConnection":
```

```
    "Server=YOUR_SERVER_NAME;Database=ASPNetCoreCRUDDb;Trusted_Connection=True  
    ;TrustServerCertificate=True;"
```

```
}
```

 Replace YOUR\_SERVER\_NAME with:

- (localdb)\\MSSQLLocalDB  
OR
  - Your SQL Server instance name
- 

## Step 6: Register DbContext in Program.cs

Open Program.cs and add:

```
using Microsoft.EntityFrameworkCore;
```

```
using StudentManagementApp.Data;
```

```
var builder = WebApplication.CreateBuilder(args);
```

```
// Add services to the container.
```

```
builder.Services.AddControllersWithViews();
```

```
// Register DbContext
```

```
builder.Services.AddDbContext<ApplicationDbContext>(options =>
```

```
options.UseSqlServer(
```

```
builder.Configuration.GetConnectionString("DefaultConnection"));
```

```
var app = builder.Build();
```

```
app.UseHttpsRedirection();
```

```
app.UseStaticFiles();
```

```
app.UseRouting();
```

If you encounter an error when adding migration, move this block up (just below the builder declaration)

builder  
declaration

```
app.MapControllerRoute(  
    name: "default",  
    pattern: "{controller=Home}/{action=Index}/{id?}");  
  
app.Run();
```

---

## ● Step 7: Create Database Using Migrations

Open **Package Manager Console**

Run:

Add-Migration Initial

Then:

Update-Database

### ✓ Verify in SQL Server Management Studio

- Open SSMS
- Connect to your server
- Check database:

ASPNetCoreCRUDDb

- Verify Students table exists
- 

## ● Step 8: Scaffold Controller and Views

1. Right-click **Controllers**
2. Click **Add → Controller**
3. Select:

MVC Controller with views, using Entity Framework

4. Model class → Student
5. Data context → ApplicationDbContext

6. Click **Add**

 **This Automatically Creates:**

- StudentsController
  - Views:
    - Index
    - Create
    - Edit
    - Details
    - Delete
- 

## **Step 9: Understand Generated Views**

### **Index View**

- Displays student records in table
- Contains buttons:
  - Edit
  - Details
  - Delete

### **Create View**

- Form to add new student

### **Edit View**

- Modify student record

### **Details View**

- View full student info

### **Delete View**

- Confirmation page before deleting

 Notice:

- Labels use [Display(Name="...")]
  - Required fields enforce validation automatically
- 

 **Step 10: Add Navigation Link**

Open:

Views/Shared/\_Layout.cshtml

Inside <ul class="navbar-nav">, add:

```
<li class="nav-item">

    <a class="nav-link text-dark" asp-controller="Students" asp-action="Index">

        Manage Students

    </a>

</li>
```

Now you can access the Students module from the navigation menu.

---

 **Step 11: Test CRUD Operations**

Run the application:

Ctrl + F5

Go to:

Manage Students

 **Perform:**

 **Create**

- Add a new student

 **Edit**

- Modify student details

### Details

- View complete record

### Delete

- Remove student
- 

### Verify Changes in SQL Server

After each operation:

1. Refresh Students table in SSMS
  2. Confirm changes are reflected immediately
- 

### What You Accomplished

You successfully:

- Created ASP.NET Core MVC project (.NET 9)
  - Built a model with data annotations
  - Configured Entity Framework Core
  - Connected to SQL Server
  - Created and applied migrations
  - Scaffolded controller and views
  - Performed full CRUD operations
-